CSCI 432/532, Spring 2026

Homework 7

Due Monday, March 9 at 8:30am

Submission Requirements

- Type or clearly hand-write your solutions into a PDF format so that they are legible and professional. Submit your PDF on Canvas or Gradescope.
- Put each **sub**problem on a separate page and select the corresponding problem via the Gradescope interface.
- Do not submit your first draft. Type or clearly re-write your solutions for your final submission. If your submission is not legible, we will ask you to resubmit.

Collaboration and Resources

The *best* resources for completing the homework are (in no particular order):

- Your own notes from lectures and problem sessions, and/or lecture recordings.
- The lecture notes linked from the course website for the lecture day.
- The questions channel on Discord.
- Office hours.
- Discussions with classmates.

You may also access almost any resource in preparing your solution to the homework. The exception is that you may not seek answers to the problems on the homework directly, such as by pasting them into a GenAI tool, searching for solutions on a search engine, looking for them on Chegg or similar websites, or asking another person for the solution.

Additionally, you ***must***

- Write your solutions in your own words—this means that you should never be *copying* anything of substance from any source, and
- credit every resource you use, including GenAI tools, by providing links or full chat transcripts. If you use the provided LaTeX template, you can use the `Sources` environment for this. Ask if you need help!

> Remember, if you choose to use GenAI tools, not only must you provide a full transcript of your conversation (or a link to one), you must also use the following prompt (or some variation of it), and provide a full transcript of the conversation.

Prompt:

You are acting as a teaching assistant for an advanced undergraduate/graduate algorithms and computer science theory course.
Your role is not to provide full solutions or final answers.
Instead, you should act like we are having a conversation. Ask me a single question and let me respond. Avoid asking me many questions at once or giving me a lot of information at one time. Guide me using hints, leading questions, partial insights, and sanity checks. Help me debug my own proofs, algorithms, or reductions. Do not give a complete solution or full proofs. Be patient with me. Don’t give me details so that I can move on to the next part of a problem. Make sure that I understood before moving on.
The course uses Jeff Erickson’s Models of Computation lecture notes, so you should guide me to answer problems using the definitions, notation, and style from those notes.
Assume I am a serious student trying to learn, not to shortcut the assignment. I am attaching the assignment, so you should refuse to provide a complete solution to the problems listed. Of course, you can walk me through the "Solved Problems" listed at the end if I ask.

1. Recall the language NEVERREJECT $= \{\langle M \rangle : \text{REJECT}(M) = \emptyset\}$ from last homework. Prove that NEVERREJECT is undecidable via ***reduction***.

2. Let $\langle M \rangle$ denote the string encoding of a Turing machine M. Recall that if w is a string, then ww is a new string that repeats w twice. Prove that the following language is undecidable.

$$\text{SELFREPEATACCEPT} := \{\langle M \rangle \mid M \text{ accepts the string } \langle M \rangle \langle M \rangle\}$$

Undecidability proof rubric.

- **Diagonalization**
  - + 4 for a correct wrapper Turing machine
  - + 6 for self-contradiction proof (= 3 for $\Longleftarrow$ + 3 for $\Longrightarrow$)
- **Reduction**
  - + 4 for a correct reduction
  - + 3 for "if" proof
  - + 3 for "only if" proof

3. Recall that a ***vertex cover*** of a graph is a set of a vertices that touches every edge of that graph. Suppose you are given a magic black box that somehow answers the following decision problem *in polynomial time*:

   - INPUT: an undirected graph G and a positive integer k.
   - OUTPUT: TRUE if G has a vertex cover of size k and FALSE otherwise.

   (a) Using this black box as a subroutine, describe an algorithm that solves the following ***optimization problem*** *in polynomial time*:
   - INPUT: an undirected graph G.
   - OUTPUT: the size of the smallest vertex cover of G.

   (b) Using this black box as a subroutine, describe an algorithm that solves the following ***search problem*** *in polynomial time*:
   - INPUT: an undirected graph G.
   - OUTPUT: a vertex cover in G of smallest size.

   Unlike in the problem session solutions, you do not need to argue the correctness of your algorithms. But you still do need to argue that they run in polynomial time.

4. Formally, a ***proper coloring*** of a graph $G = (V, E)$ is a function $c : V \to \{1, 2, \ldots, k\}$, for some integer k, such that $c(u) \neq c(v)$ for all $uv \in E$. Less formally, a valid coloring assigns each vertex of G a color, such that every edge in G has endpoints with different colors. The ***chromatic number*** of a graph is the minimum number of colors in a proper coloring of G.

   Suppose you are given a magic black box that somehow answers the following decision problem *in polynomial time*:

   - INPUT: An undirected graph G and an integer k.
   - OUTPUT: TRUE if G has a proper coloring with k colors, and FALSE otherwise.

Using this black box as a subroutine, describe an algorithm that solves the following ***search problem*** *in polynomial time*:

- INPUT: An undirected graph G.
- OUTPUT: A valid coloring of G using the minimum possible number of colors.

[Hint: You can use the magic box more than once. The input to the magic box is a graph and ***only*** *a graph, meaning* ***only*** *vertices and edges.]*

Unlike in the problem session solutions, you do not need to argue the correctness of your algorithms. But you still do need to argue that they run in polynomial time.

Basic reduction (algorithms). 10 points =

+ 4 for calling the magic black box as a subroutine in the algorithm. No points here if it is not called on the correct input data type.

+ 2 for a correct algorithm.

+ 2 for an algorithm that runs in polynomial time (assuming the black box runs in polynomial time).

+ 2 for a valid argument that the algorithm runs in polynomial time.

Solved problems

1. Consider the language SometimesHalt $= \{\langle M \rangle \mid M \text{ halts on at least one input string}\}$. Note that $\langle M \rangle \in$ SometimesHalt does not imply that M *accepts* any strings; it is enough that M *halts* on (and possibly rejects) some string.

   Prove that SometimesHalt is undecidable.

Solution: We can reduce the standard halting problem to SometimesHalt as follows:

```
DecideHalt(⟨M, w⟩):
  Encode the following Turing machine M′:
    M′(x):
      (ignore x)
      run M on input w
  return DecideSometimesHalt(⟨M′⟩)
```

We prove this reduction correct as follows:

$\Longrightarrow$ Suppose M halts on input w.
Then M' halts on *every* input string x.
So DecideSometimesHalt must accept the encoding $\langle M' \rangle$.
We conclude that DecideHalt correctly accepts the encoding $\langle M, w \rangle$.

$\Longleftarrow$ Suppose M does not halt on input w.
Then M' diverges on *every* input string x.
So DecideSometimesHalt must reject the encoding $\langle M' \rangle$.
We conclude that DecideHalt correctly rejects the encoding $\langle M, w \rangle$.

■